Web-Based Structural Analysis Program Using OpenSees

Abstract

OpenSees (Open System for Earthquake Engineering Simulation) is an open-source software framework designed to model and simulate the performance of structural and geotechnical systems subjected to earthquakes. It offers advanced capabilities for modeling nonlinear responses using diverse material models, elements, and solution algorithms. To use OpenSees, users must set up the appropriate environment and be proficient in either Tcl or Python. Visualization often requires additional libraries, such as OPSVIS, which may have compatibility issues across different versions.

Many existing software tools, such as FeView and OpenSeesPyView, face compatibility challenges across operating systems, require local installation, and demand careful version control. Additionally, some commercial software, like STKO, can be costly.

This project presents the development of a web-based Structural Analysis Program that utilizes OpenSees as a computational backend for structural engineering applications. The web application addresses the accessibility limitations of traditional desktop-based structural analysis software by providing a browser-based interface. This approach eliminates the need for local OpenSees installation and reduces the necessity of learning multiple tools and frameworks, such as Tcl, Python, and various plotting libraries.

Currently, the program is developed for linear structural analysis, making it particularly well-suited for teaching undergraduate structural analysis courses and supporting rapid parametric studies.

The system architecture consists of three primary components: Model module, Analysis module, and Results module, all integrated with a Python backend. The Model module enables users to define structural components following OpenSees conventions, including nodes, supports, materials, sections, transformations, integrations, elements, time series, and patterns. The Analysis module provides configuration controls for OpenSees analysis parameters such as constraints, numberers, system solvers, algorithms, integrators, analysis types, and recorders. The Results module delivers comprehensive visualization of structural responses, including model geometry, deflections, and internal forces, through interactive charts and animations using D3.js.

The backend infrastructure employs a Python Flask framework that orchestrates OpenSeesPy operations, translating user inputs into OpenSees command sequences, executing finite element analyses, and processing output data for client consumption. Communication between the frontend and backend occurs through RESTful API endpoints that handle JSON data exchange, enabling real-time analysis execution and results retrieval. The system supports the analysis of various structural types, including beams, frames, and trusses, with extensibility for additional element types and analysis procedures.

This web-based approach significantly reduces barriers to structural analysis by providing immediate access to OpenSees capabilities without requiring specialized software installation or configuration. The platform facilitates collaborative engineering workflows, enables rapid parametric studies, and supports educational applications in structural and earthquake

engineering. The integration of OpenSees computational capabilities with modern web technologies demonstrates the potential for cloud-based structural analysis tools to enhance accessibility and collaboration in engineering practice, while maintaining the robust analytical capabilities required for professional applications.

Timeline

Week & Dates	Objectives	Key Deliverables & Locations
Week 1 (May 15–18)	**Foundation & Setup**	**OpenSeesPy Fundamental:** Completed basic OpenSeesPy tutorials and example scripts **Architecture design:** Defined folder structure and module boundaries **OpenSees Runner.py:** JSON → OpenSees commands translator, analysis executor & recorder **Flask API:** Exposed endpoints for model submission and result retrieval **Template Json:** Comprehensive command template covering nodes, elements, loads, supports
Week 2 (May 19–25)	**Model Definition API**	**OpenSees Service:** HTTP client to invoke Flask model endpoints **Model Builder UI:** Interactive command palette (ModelBuilderPage.jsx + components/model-builder/) **Command converter:** Transforms user inputs into OpenSees command objects (in modelUtils.js) **State store:** Persist model definitions via useModelStore.js
Week 3 (May 26–June 1)	**Common Data Handling Utilities**	**Load parser (plotParser.js):** Extracts and normalizes node/element load definitions **Recorder parser (recorderParsing.js):** Maps OpenSees recorders to client-side data structures **General helpers:** modelUtils.js (geometry & command helpers) + idGenerator.js (unique ID schemas)
Week 4 (June 2–8)	**Canvas & Interactive Visualization**	**Infinite grid & axes:** Implemented background grid with ± axes in Model.jsx **D3 refinements:** Synchronized arrow colors with element strokes; enabled smooth zoom & pan controls **Visualization integration:** Rendered models, supports, and applied loads in dedicated Visualization components
Week 5 (June 9–15)	**Analysis & Results Delivery**	**Analysis page:** Built AnalysisPage.jsx with parameter controls; wired into useAnalysisStore.js **Results page:** Created ResultsPage.jsx with interactive charts & animations under components/visualization/ **API polishing:** Finalized and tested all endpoints in Frontend/src/api/; ensured robust error handling and JSON schema validation